

Confidential Computing Hardware Roots of Trust

Pr. Valerio Schiavoni

University of Neuchâtel

Valerio.Schiavoni@unine.ch

Tentative Agenda of the Course (v.1)

1. Introduction & Confidential Computing Foundations

2. Hardware Root of Trust

3. TEE Fundamentals

4. Intel SGX Deep Dive

5. AMD SEV, SEV-ES, SEV-SNP

6. Intel TDX

7. Confidential Containers

This schedule
can change!

Some chapters
can be merged
or removed.

Agenda

- **Understand Hardware Roots of Trust:**
 - how Secure Boot and TPMs establish a foundational layer of trust
- **Explore Attestation:**
 - how systems prove their integrity using trusted hardware
- **Learn Secure Key Provisioning:**
 - how cryptographic secrets are safely delivered to trusted environments

Lecture objectives

- Explain the roles of **Secure Boot** and TPMs, in establishing trust from power-on
- Understand the **attestation process** and how it supports remote verification of platform integrity
- Describe how **secure key provisioning** enables trusted computing by binding secrets to verified hardware states

Course Outcomes

- After this chapter, you should be able to:
- **Identify the key components** and functions of Secure Boot and TPMs
- Describe **how attestation works** and interpret a basic attestation flow
- Explain how cryptographic **keys are provisioned securely** using hardware-based trust anchors

Section I

Secure Boot, TPMs, and fTPMs

What is a “Root of Trust”?

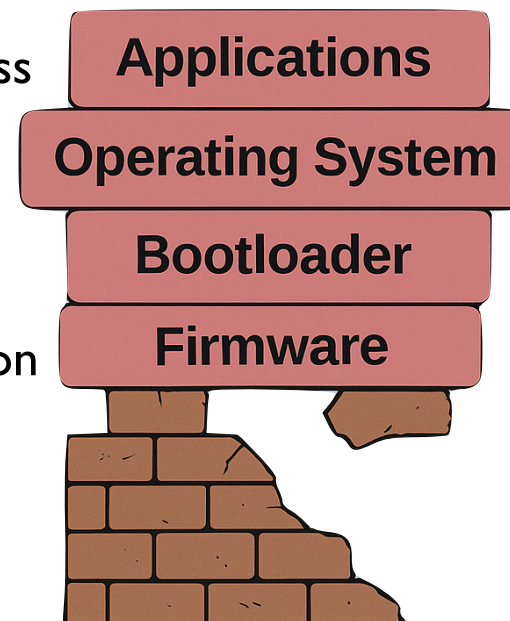
- Definition
 - A **Root of Trust** (RoT) is a trusted hardware or firmware component that serves as the secure foundation for all other system operations.
- Why it matters
 - It provides the **first link** in the trust chain:
 - Ensures that system components (e.g., OS, apps) start from a known good state
 - Enables secure boot, attestation, and key protection

“Ground zero” of security — if the root is compromised, all higher-level protections are meaningless.

Why software-only trust is not enough?

PUF-based
software RoT?

- Software is mutable
 - Bootloaders, OS kernels, and apps can be replaced or modified
 - Even signed software can be loaded after a compromised boot process
- Admins $\neq$ always trustworthy
 - Insider threats, malware, and misconfigurations can bypass software defenses
 - “Root access” in software gives control over everything
- No anchor without hardware
 - Software-only solutions lack a tamper-resistant foundation
 - There’s no way to prove early-stage integrity without hardware-backed measurement

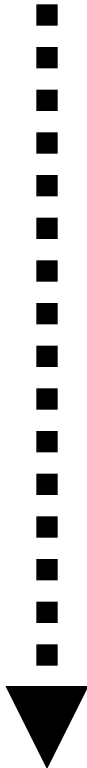


Without hardware roots of trust, a compromised system can fake being secure — and you’d never know.

Secure Boot – What it protects

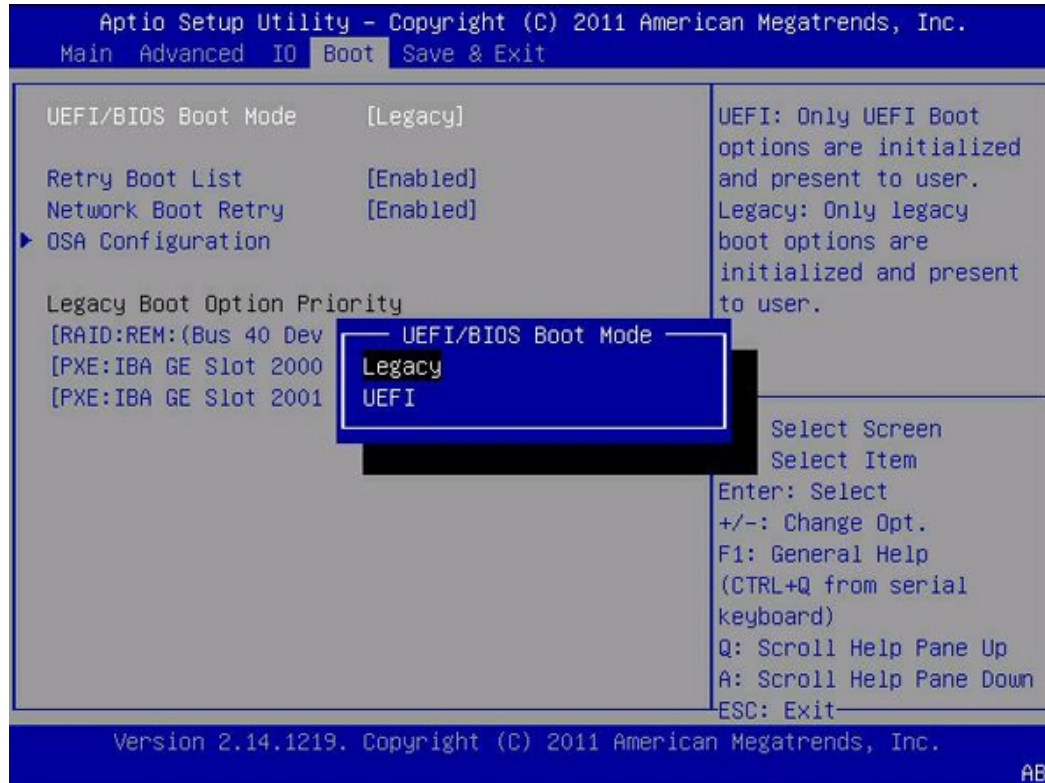
- Secure Boot ensures that a system only boots code that is signed and trusted, starting from the firmware up to the OS.
- What it protects against:
 - Bootkit and rootkit attacks
 - Tampered firmware or bootloaders
 - Unauthorized OS kernels or drivers
- How it works:
 - Firmware checks **digital signature** of the bootloader
 - Bootloader checks the OS kernel
 - Only signed and approved components are allowed to load
- Goal:
 - Establish a “**chain of trust**” that starts with hardware and extends to the OS.

Secure Boot: Chain of trust

- 
- How the chain works:
 - 1. Hardware/ROM
 - Executes immutable firmware
 - Starts the chain with known-good root of trust
 - 2. Firmware (UEFI/BIOS)
 - Verifies and launches a **signed bootloader**
 - 3. Bootloader
 - Verifies and loads a signed OS kernel
 - 4. Operating System Kernel
 - Optionally verifies signed drivers and services
 - Each step verifies the next
 - If any link is broken (e.g., unsigned or tampered code), the system halts or enters recovery mode

Goal: Ensure the system starts from a trusted state — every time.

Legacy Boot UEFI vs Secure Boot



- Legacy Boot (BIOS)
 - Executes the bootloader without verifying its integrity
 - No cryptographic checks → any code can run at boot
 - Vulnerable to bootkits, rootkits, and stealth malware

Legacy Boot: Troubles

- No signature verification: legacy BIOS does not verify digital signatures on the bootloader, kernel, or drivers.
 - Any code—malicious or not—can be loaded at boot without detection.
- No chain of trust:
 - Legacy boot simply loads the first sector of the boot device (MBR or equivalent).
 - There's no mechanism to validate the integrity of the bootloader or OS.
 - Malware can hook into the boot process and persist **below the OS level**.
- Rootkits and Bootkits:
 - Attackers can install stealth malware that runs before the OS loads
 - “Membromi”

Researchers find attack on Millions of printers

Can a hacker take control of your printer? Using it to sniff information from the network, steal confidential information, or even attack other machines. Researchers have found an attack impacting millions of printers around the world.

Is Mebroot the stealthiest Rootkit in the world?

Federal agents raided unnamed operators of the Rustock "botnet" vast network of computers around the globe infected with malicious software that allows distribution of huge volumes of spam.

Link Discovered Between TDSS Rootkit and DNSChanger Trojan

TDSS rootkit, the sophisticated and difficult to remove malware behind many advanced attacks also appears to have helped spread the DNSChanger Trojan.

Researcher finds attack on Apple battery firmware. [Blackhat 2011]

The firmware used to control the charging of Apple's laptop batteries could be attacked by malware. Allowing the attacker to potentially override safety mechanism which could lead to an attack.

Advanced Persistent Attacks: BIOS Rootkit -"Mebromi"

Hamza Sirag, Nihant Bondugula, Rishabh Gupta
Graduate School of Computer Science, George Mason University, Fairfax, VA

1. Abstract

As cyberspace has evolved malware has also evolved. According to the United States Computer Emergency Readiness Team, malware is defined as malicious software that consists of programming (code, scripts, active content, and other forms) designed to damage, exploit, or

vulnerabilities associated with Mebromi, the tools that take advantage of those technological vulnerabilities, mitigation of the technological vulnerabilities, future of advanced persistent attacks, future of BIOS targeting, and provide a conclusion summarizing our research.

DE MYSTERIIS DOM JOBSIVS: EFI ROOTKITS

SNARE
@ SYSCAN SINGAPORE
APRIL 2012



assurance

Legacy Boot UEFI vs Secure Boot

- UEFI Secure Boot
 - Requires that all boot components are signed with trusted keys
 - Firmware verifies bootloader signature before execution
 - Blocks unauthorized or tampered code from loading

Feature	Legacy	UEFI
Signature checks	✗	✓
Boot-time integrity	✗	✓
Malware resistance	✗	✓

Case Study: **LoJax** – First UEFI/BIOS Rootkit in the Wild

- What Is **LoJax**?
 - A persistent bootkit discovered in 2018
 - Injected malicious code into the SPI flash containing the BIOS/UEFI firmware
 - Survives OS reinstallations, disk wipes, and antivirus scans
- Could install spyware, keyloggers, or remote access tools
- Used by APT28 (Fancy Bear ) for espionage against government targets
 - *Advanced Persistent Threat*
- Exploited lack of signature checks in BIOS systems

Case Study: **LoJax** – First UEFI/BIOS Rootkit in the Wild

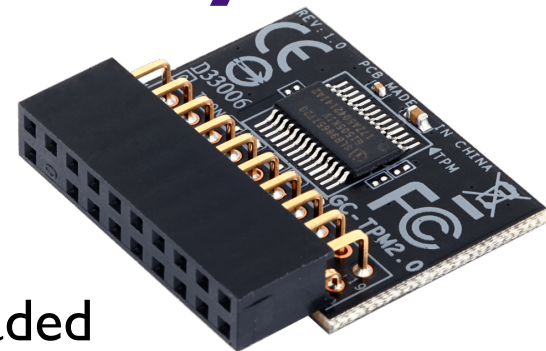
- Why secure boots blocks LoJax ?
- Signature Verification of Bootloader
 - LoJax injects a malicious bootloader to gain control early in the boot process.
- Secure Boot verifies digital signatures on the bootloader against a trusted key database (**UEFI DB**).
- If the injected bootloader is unsigned or altered, boot is halted.

where?

Case Study: **LoJax** – First UEFI/BIOS Rootkit in the Wild

- Tampered Kernel/Driver would also be rejected
 - If LoJax tries to load a malicious kernel or driver, Secure Boot will prevent it unless the code is signed with a trusted certificate
- Enforced Chain of Trust
 - Secure Boot ensures that every stage (UEFI → bootloader → OS kernel) is verified.
 - **LoJax's** bootkit breaks that chain, so Secure Boot detects and blocks the boot process.
-  Limitations
 - Secure Boot doesn't stop the firmware from being modified (e.g., if SPI flash is writable).
 - But if the attacker's goal is to run custom code early, Secure Boot blocks it unless they sign it with a trusted key (which they can't).

TPM vs BIOS/UEFI security

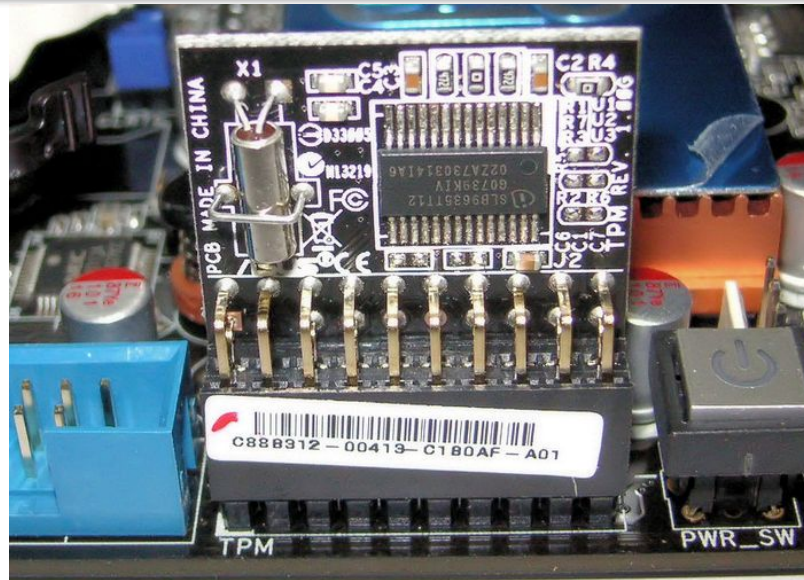


- BIOS/UEFI Secure Boot alone...
 - Verifies signed bootloaders and OS
 - But doesn't **record or report** what was loaded
 - Offers no remote visibility or audit trail of system state
- What a TPM adds: **Trusted Platform Module**
 - Cryptographically measures each boot step (e.g., firmware, bootloader, kernel)
 - Stores measurements in Platform Configuration Registers (PCRs)
 - Supports **attestation** — remote proof of system integrity
 - Can seal secrets to a known-good boot state (e.g., only release keys if PCR values match)

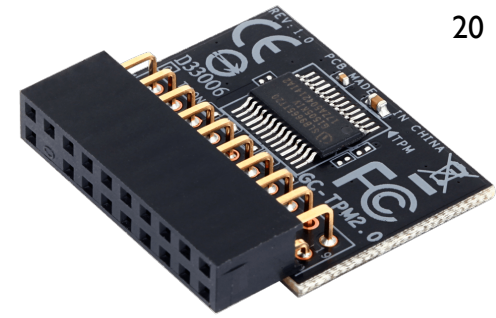
UEFI + TPM

- UEFI Secure Boot enforces trust at boot time
- TPM enables auditability, remote trust, and conditional access based on boot state

TPM extends BIOS/UEFI security from “verify once” to “verify + report + enforce.”



What is a TPM?



- A Trusted Platform Module a dedicated hardware chip that provides secure, tamper-resistant services for **platform integrity, key storage, and attestation.**
- Key features:
 - Secure key generation and storage
 - Cryptographic operations (signing, encryption, sealing)
 - Measurement and logging of system boot events (into PCRs)
 - Remote attestation: prove the device booted into a trusted state
- Enables policies like: *“only unlock secrets if the system is unmodified.”*

TPM: a tiny, tamper-resistant vault with a notebook inside that records your boot history

Features of a commercial TPM module

- Performs secure cryptographic operations directly **on-chip** (e.g., RSA/ECC signing, hashing)
- Tamper-resistant design prevents external software from accessing keys
- Generates and stores keys securely (SRK, EK, Attestation Keys)
- Low throughput: 1000's of crypto operations per minute,
 - Sufficient for key attestation and sealing tasks (not bulk encryption)
- Stores boot integrity data in Platform Configuration Registers (PCRs) via chained hashing of firmware and boot components
- Generates signed quotes containing PCR values for remote attestation

TPM Core Functions

1. Key storage

- Generates and stores cryptographic keys in **secure non-volatile memory**
- Keys never leave the TPM in plaintext
- Supports RSA, ECC, HMAC, AES (via TPM 2.0)

2. Measurement (Integrity Logging)

- Measures software/firmware loaded at boot (e.g., BIOS, bootloader)
- Stores hashes in Platform Configuration Registers (PCRs)
- Used in remote attestation and measured boot

3. Sealing

- Binds secrets (e.g., encryption keys) to a specific hardware state
- Secret is only unsealed if PCR values match the known good boot configuration
- Enables policies like: “only decrypt if system integrity is intact”

The TPM acts as both a secure vault and a trust anchor for platform integrity.

TPM Internals

- **PCRs (Platform Configuration Registers)**

- Immutable hash registers used to record measurements of boot events
- Values are extended (not overwritten):
 - $\text{PCR} = \text{SHA256}(\text{PCR} \parallel \text{new_measurement})$
- Used for measured boot, sealing, and attestation

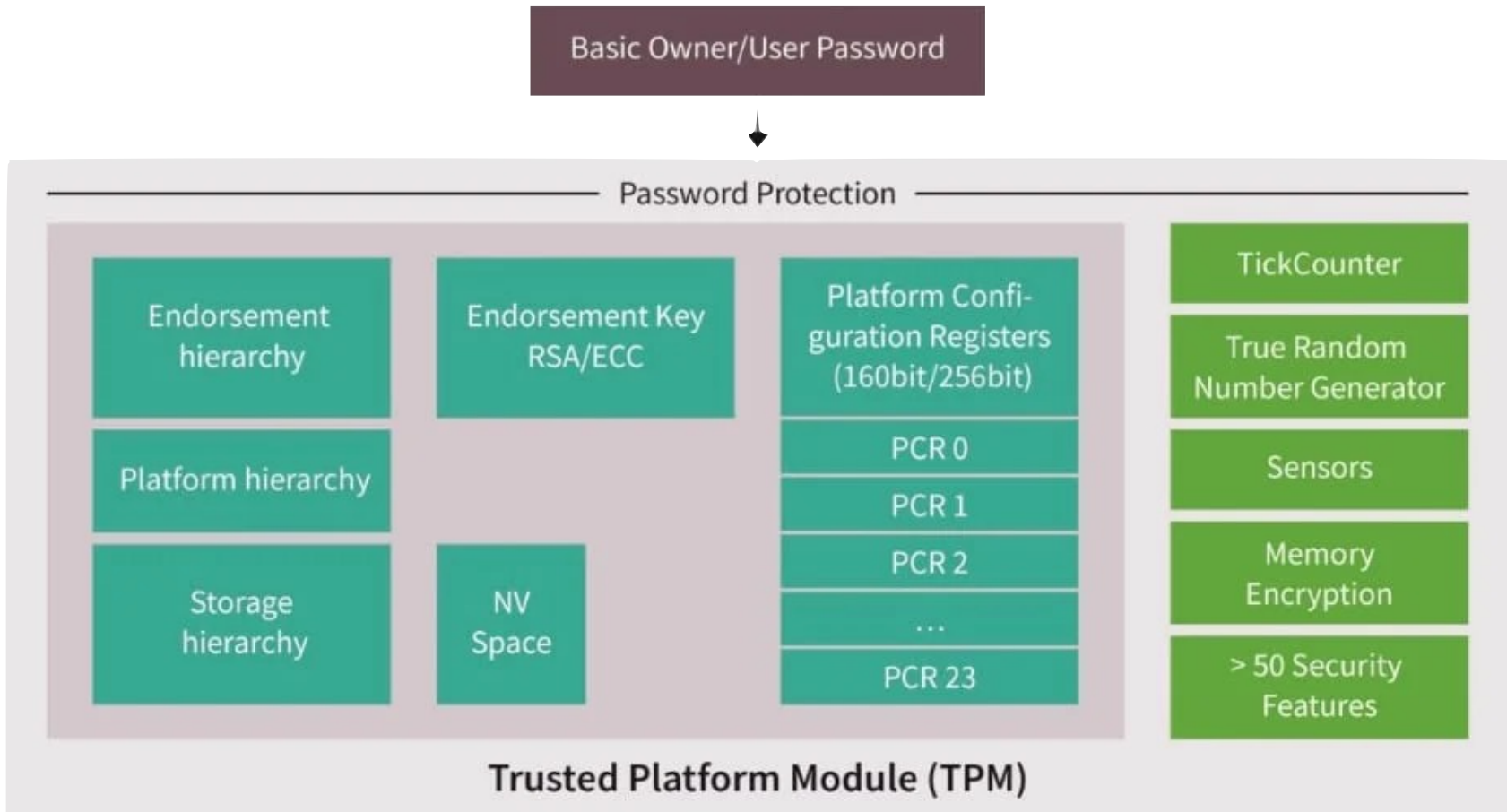
- **NVRAM (Non-Volatile RAM)**

- Stores small amounts of persistent configuration data
- Can hold keys, policies, counters, or logs
- Protected by access control policies (e.g., PIN, auth)

- **Endorsement Key (EK)**

- **Burned** into the TPM at manufacturing
- Unique, immutable identity of the TPM
- Used to sign quotes during attestation
- Issued and verifiable by **vendor CA** (e.g., Intel, Infineon)

Example: OPTIGA TPM SLM 9670



- Retail price: 2.5 EUR

PCRs explained (1/3)

- What are PCRs?
 - Special **registers** inside the TPM used to store hash values of system components measured during boot
 - PCRs are **write-protected** — values can only be extended, not replaced
- New measurements are appended via hashing:
 - $\text{PCR}_n = \text{SHA256}(\text{PCR}_n \parallel \text{new_value})$
 - This creates a cumulative log that reflects everything loaded up to that point
- Why PCRs matter
 - Used in measured boot to track BIOS → Bootloader → Kernel → Drivers
 - Used in sealing: secrets can only be unsealed if PCRs match the trusted state
 - Used in attestation: remote parties verify system integrity via PCR quote

PCRs as a “digest history” of your machine’s boot path — any tampering causes a mismatch.

PCRs explained (2/3)

- How many PCRs are there?
 - TPM 1.2:
 - Has 24 PCRs (numbered 0–23)
 - Limited to SHA-1 (160-bit hashes)
 - TPM 2.0:
 - Also typically provides 24 PCRs, but with configurable hashing algorithms (e.g., SHA-1, SHA-256, SHA-384)
 - Each PCR bank corresponds to a hash algorithm
 - So: 24 PCRs × N hash algorithms = multiple banks (e.g., SHA-1 bank, SHA-256 bank, etc.)

PCRs explained (3/3)

- How large is a PCR ? It depends.

Algorithm	PCR size
SHA-1	20 bytes (160 bits)
SHA-256	32 bytes (256 bits)
SHA-384	48 bytes (384 bits)

- All PCRs in a given bank use the same size, based on the that bank's hash
- Usage examples:
 - PCR 0–7: typically used for BIOS, bootloader, kernel
 - PCR 8–15: for OS, config, and application-level data
 - PCR 16–23: for debugging, testing, or vendor-defined use

fTPM vs dTPM:

Firmware vs Discrete TPM

- Firmware TPM (fTPM)
 - Implements TPM functionality in firmware (e.g., inside the system's CPU or chipset)
 - Uses Trusted Execution Environment (TEE) (e.g., ARM TrustZone or AMD PSP)
 - More cost-effective and space-saving
 - Present in most modern consumer devices (e.g., laptops with Windows 11)
- Discrete TPM (dTPM)
 - A separate physical chip soldered onto the motherboard
 - Offers stronger isolation and tamper-resistance
 - Common in enterprise-grade laptops, servers, and security-critical devices



Example: BitLocker+TPM



- Microsoft's full-disk encryption feature
- Goal: protect data at rest — i.e., when the device is powered off or stolen.
- BitLocker integrates with a TPM to secure encryption keys and ensure the integrity of the system at boot
 - It stores the volume encryption key (VEK) sealed inside the TPM.
 - The key is released only if the system's boot measurements (PCR values) match expectations.
 - TPM logs and verifies boot components (UEFI, bootloader, etc.) during startup.
 - If any component is altered (e.g., due to malware), the TPM refuses to release the key, and BitLocker enters recovery mode.

Example: Measured Boot+TPM

- Each boot stage (firmware, bootloader, OS, drivers) is measured and logged into TPM PCRs
- Security software (e.g., Microsoft Defender, Intune, Microsoft Entra ID -IAM) can verify these measurements remotely
- Ensures the system booted into a known-good state
- Foundation for remote attestation and conditional access



Example: TPM in Linux

- Measured Boot with TPM and `tpm2-tools`
- Linux distros like Ubuntu, Fedora, and Debian support measured boot via:
 - GRUB2, `initrd`, `systemd-boot`, and the IMA (Integrity Measurement Architecture - since kernel 2.6.30)
 - <https://ima-doc.readthedocs.io/en/latest/ima-concepts.html>
 - Boot components are measured into PCRs using `tpm2_pcrread`, `tpm2_quote`, and logged via `tpm2_eventlog`

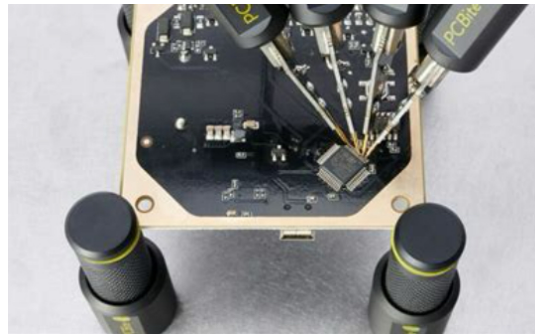
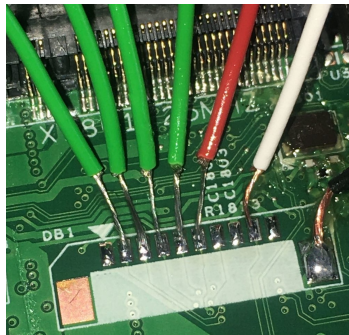
```
ubuntu@rigi-2:~$ tpm2
Display all 101 possibilities? (y or n)
tpm2
tpm2_activatecredential
tpm2_certify
tpm2_certifycreation
tpm2_changeauth
tpm2_changeeps
tpm2_changepps
tpm2_checkquote
tpm2_clear
tpm2_clearcontrol
tpm2_clockrateadjust
tpm2_commit
tpm2_create
tpm2_createtask
tpm2_createek
tpm2_createpolicy
tpm2_createprimary
tpm2_dictionarylockout
tpm2_duplicate
tpm2_ecdhkeygen
tpm2_ecdhzgen
tpm2_ecephemeral
tpm2_encryptdecrypt
tpm2_eventlog
tpm2_evictcontrol
tpm2_flushcontext
tpm2_getcap
tpm2_getcommandauditdigest
tpm2_geteccparameters
tpm2_getekcertificate
tpm2_getpolicydigest
tpm2_getrandom
tpm2_getsessionauditdigest
tpm2_gettestresult
tpm2_gettime
tpm2_hash
tpm2_hierarchycontrol
tpm2_hmac
tpm2_import
tpm2_incrementalselftest
tpm2_load
tpm2_loadexternal
tpm2_makecredential
tpm2_nvcertify
tpm2_nvdefine
tpm2_nvextend
tpm2_nvincrement
tpm2_nvread
tpm2_nvreadlock
tpm2_nvreadpublic
tpm2_nvsetbits
tpm2_nvundefine
tpm2_nvwrite
tpm2_nvwriteblock
tpm2_pcrallocate
tpm2_pcrevent
tpm2_pcrextend
tpm2_pcrread
tpm2_pcrreset
tpm2_policyauthorize
tpm2_policyauthorizenv
tpm2_policyauthvalue
tpm2_policycommandcode
tpm2_policycountertimer
tpm2_policycphash
tpm2_policyduplicationselect
tpm2_policylocality
tpm2_policynamehash
tpm2_policynv
tpm2_policynvwritten
tpm2_policyor
tpm2_policypassword
tpm2_policypcr
tpm2_policyrestart
tpm2_policysecret
tpm2_policysigned
tpm2_policytemplate
tpm2_policyticket
tpm2_print
tpm2_quote
tpm2_rc_decode
tpm2_readclock
tpm2_readpublic
tpm2_rsadecrypt
tpm2_rsaencrypt
tpm2_selftest
tpm2_send
tpm2_sessionconfig
tpm2_setclock
tpm2_setcommandauditstatus
tpm2_setprimarypolicy
tpm2_shutdown
tpm2_sign
tpm2_startauthsession
tpm2_startup
tpm2_stirrandom
tpm2_testparms
tpm2_unseal
tpm2_verifysignature
tpm2_zgen2phase
```

TPM Limitations

- Performance Constraints 
 - Not designed for high-throughput crypto
 - Operations like signing, key generation, and attestation are relatively slow (**few ms/op**)
 - Not suitable for bulk encryption or real-time workloads
- Abstraction & Complexity
 - TPM APIs are low-level and non-intuitive (e.g., TPM 2.0 spec is over 1,000 pages)
 - Debugging TPM errors can be opaque and platform-specific
- Lack of transparency:
 - Firmware TPMs (fTPMs): vendor-specific closed implementations
 - Difficult/impossible to audit or verify behavior
 - Hardware bugs (e.g., **TPM sniffing**, fTPM rollback) are hard to detect and patch

TPM Sniffing

- An attacker with physical access can attach a logic analyzer to TPM communication lines
- I²C, LPC, or SPI buses between the CPU and a discrete TPM are often not encrypted
- What can be leaked?
 - Key material, PIN attempts, sealed/unsealed secrets, or PCR quote data
 - Even if the TPM itself is secure, the bus traffic may be in plaintext
- Mitigations
 - Use firmware TPMs (fTPMs) to avoid exposed buses
 - Opt for integrated TPMs with hardened pathways



TPM in the Cloud - vTPM

- A software-emulated TPM exposed to cloud VMs
- Backed by a physical TPM on the host or a software implementation
- Available in cloud platforms like Azure, AWS Nitro Enclaves (some form of TrustZone), and GCP Shielded VMs (virtualised TEEs)
- Can we trust it ?
 - vTPMs depend on the integrity of the hypervisor and cloud host OS
 - A malicious or compromised hypervisor could:
 - Forge PCR values
 - Bypass sealing restrictions
 - Mimic valid attestation flows

Trustworthy vTPM

- When combined with hardware-backed attestation (e.g., AMD SEV, Intel TDX)
- Cloud provider must offer verifiable attestation reports anchored in physical hardware
- Ideal when vTPM is used inside a TEE-backed guest (e.g., confidential VM)

future chapters

A virtual TPM is only as trustworthy as the underlying host and its attestation guarantees.

Recap: Secure Boot and TPM landscape

- Secure Boot
 - Verifies signed boot components at each stage
 - Ensures system boots into a known, trusted state
 - Prevents unsigned or tampered code from running at startup
- TPM (Trusted Platform Module)
 - Stores and protects cryptographic keys
 - Measures system integrity using PCRs
 - Enables attestation and key sealing based on hardware state
- Together:
 - Secure Boot enforces integrity ➡
 - TPM records integrity measurements and gates secrets ➡
 - Combined, they build a chain of trust from power-on to OS load

Section 2

Attestation Basics (Expanded)

What Is Attestation? (Revisited)

- Definition:
 - *Attestation is the process by which a system proves its integrity to a remote party using hardware-based measurements (e.g., via TPM).*
- Key idea 💡 :
 - A trusted component (e.g., TPM, TEE) generates a cryptographic statement—a **quote**—about the system's state (e.g., PCRs).
- Who is involved?
 - **Prover**: the system being attested (e.g., cloud VM, laptop)
 - **Verifier**: the remote party checking integrity (e.g., admin, service, key server)
 - **Trust anchor**: TPM or TEE hardware that signs measurements

Why attestation is critical?

- Enables trust decisions:
 - *Only release the key if system is untampered*
 - *Only run the workload on a clean host*
 - *Only connect to a verified peer*
- Attestation turns local hardware trust into remote verifiability.
- But...how many types of attestations exist ?

At least two: local ... (1/2)

- Local attestation:
 - Used between two enclaves or components on the same platform
 - Example: SGX enclave → SGX enclave communication
 - **Does not require certificates or external verification**
 - Faster, lower overhead
 - Goal: **Verify peer identity and enclave measurement locally**

At least two: ... and remote (2/2)

- Used to prove system integrity to an **external verifier** (e.g., a key server, client, or auditor)
- Requires cryptographic quote (signed by TPM or TEE)
- Involves certified key (e.g., Endorsement Key or Attestation Key)
- Goal: Allow **remote trust decisions** based on the system state

Local attestation for intra-system trust, and remote attestation for external verification of system integrity.

PCR values (revised)

- Platform Configuration Register (PCR)
 - A TPM register that stores the result of cumulative integrity measurements
 - Represents the state of the system during the boot process
 - Cannot be reset or overwritten during normal operation
 - Updated with:
 - $\text{PCR}_n = \text{SHA256}(\text{PCR}_n \parallel \text{new_measurement})$
- Ensures tamper-evident logging of what was loaded
- Hashing makes it computationally infeasible to forge a valid PCR value
- Reflects exact order and content of loaded components

How are PCR values used for attestation in 4 steps

1. Boot-Time Measurement

- Each component loaded during boot (e.g., BIOS, bootloader, kernel) is hashed
- The hash is extended into a specific PCR in the TPM
- This creates a cumulative, ordered record of the system's launch

2. Quote Generation

- A remote verifier challenges the system for a signed report
- TPM creates a quote: a signed statement of selected PCR values
- Quote is signed by a TPM Attestation Key (AK)

How are PCR values used for attestation in 4 steps

3.Verification:

- The verifier receives:
 - the quote (signed PCR values)
 - the AK certificate (to validate the TPM)
 - optionally, a log of measurements (event log)
- It checks:
 -  signature validity
 -  PCR values match expected good states
 -  AK is certified by a known TPM vendor (e.g., Intel, Infineon)

4.Decision

- If everything matches: “Trusted”
- If not: access is denied, key not released, or system flagged

Measured Boot Flow

- Typical Boot-Time Flow (Measured Boot):
 - 1.UEFI/BIOS
 - Loads and measures the bootloader
 - Extends measurement into PCR[0]
 - 2.Bootloader (e.g., GRUB)
 - Measures and loads the kernel image
 - Extends hash into PCR[4]
 - 3.Kernel
 - Measures and loads initramfs (initial RAM disk)
 - Extends into PCR[8]

Component	PCR Index	SHA-256 Hash (Truncated)
Bootloader	PCR[0]	9f8a...3c2d
Kernel	PCR[4]	ab12...77fe
Initramfs	PCR[8]	c4e3...b901

Values sent inside an attestation quote.

Verifier checks hashes against a **trusted known-good configuration**.

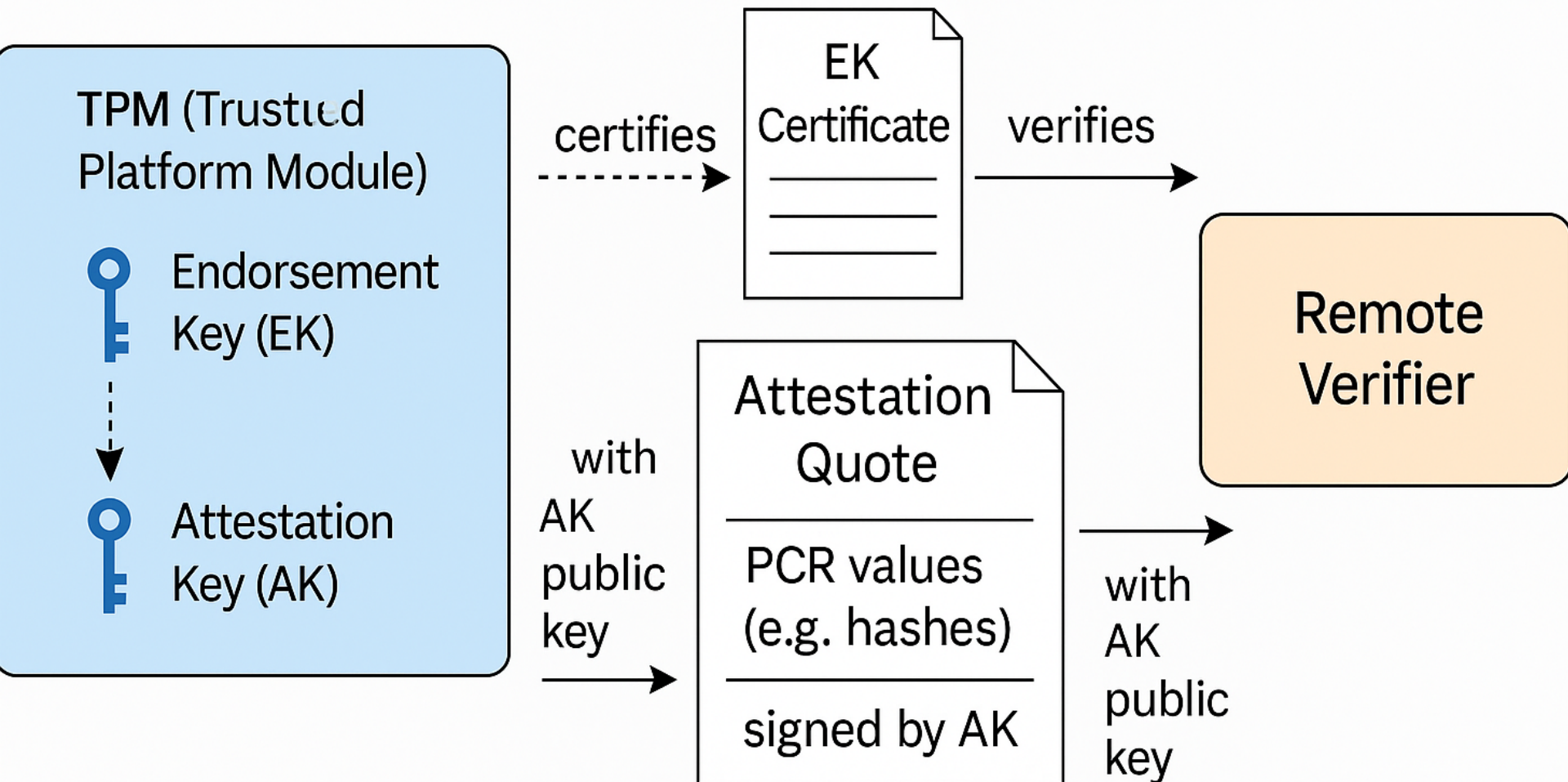
Any change will break the attestation match.

TPM vs Intel SGX Attestation

Feature	TPM	SGX
What is measured	Entire boot process (firmware → OS)	Specific enclave code
Trust anchor	TPM chip (discrete or firmware)	CPU with SGX capability
Measurement format	PCRs (hashes of boot events)	MRENCLAVE (hash of enclave binary)
Usage scope	Whole platform / OS state	Application-level / enclave-specific
Remote verification	Verifies OS and boot chain	Verifies code + identity of a secure enclave
Key exposure	Attestation key typically static (EK/AK)	Quote signed by EPID (SGX1) or ECDSA



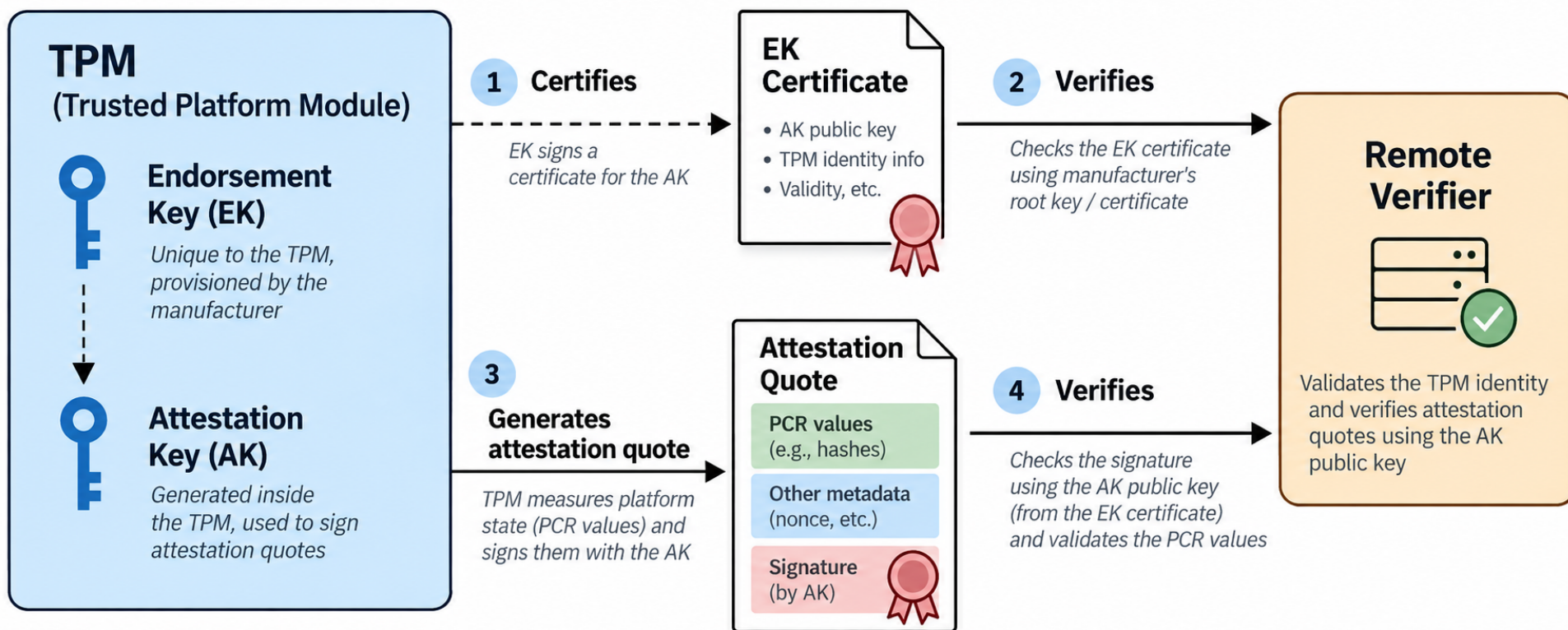
Attestation Key (AK) vs Endorsement Key (EK)



EK = Device identity. AK = Attestation signer.
They form a hardware root of trust for remote verification.



Attestation Key (AK) vs Endorsement Key (EK)



**EK = Device identity. AK = Attestation signer.
They form a hardware root of trust for remote verification.**



Attestation Key (AK) vs Endorsement Key (EK)

- Endorsement Key (EK)
 - A unique, permanent key pair burned into the TPM at manufacture
 - Acts as the **identity** of the TPM
 - EK never used directly for signing
 - Comes with an EK certificate signed by the manufacturer (e.g., Intel, Infineon)
 - Used to certify other TPM keys, like the AK
- Attestation Key (AK)
 - A non-volatile key pair created within the TPM
 - Used to sign attestation quotes (i.e., PCR values)
 - Can be certified by the EK, proving that the AK belongs to a genuine TPM
 - Typically used in remote attestation

How They Work Together

1. EK certifies the AK (→ “This key belongs to a genuine TPM”)
2. AK signs quote with PCRs
3. Verifier checks both AK certificate and signature on quote

**EK = Device identity. AK = Attestation signer.
They form a hardware root of trust for remote verification.**

Privacy Concerns

- Attestation can leak identity
 - TPM quotes are signed by keys tied to a specific hardware device
 - This risks enabling **device tracking** across services
 - **Privacy-preserving attestation** avoids linking all actions to a single identity
- Mitigations:
 1. Direct Anonymous Attestation (DAA)
 - zero-knowledge proofs: TPM validity without revealing identity
 2. Pseudonyms: unlinkable identifiers + Privacy CA
 - Auditable attestation (but not traceable)

Cloud Native Attestation:

Microsoft Azure Attestation (MAA)

- Cloud service for verifying trusted execution environments (TEEs)
- Supports:
 - Intel SGX, AMD SEV-SNP, vTPMs, and confidential VMs
- Accepts evidence (e.g., SGX quote), validates it, and returns a JSON Web Token (JWT)
- Integrates with Azure Key Vault and policy-based access control

Cloud Native Attestation: SPIFFE / SPIRE

- SPIFFE: Secure Production Identity Framework for Everyone
 - Provides identities (SVIDs) to workloads using attestation
 - <https://spiffe.io>
- SPIRE: the reference implementation
 - Verifies node or workload identity using plugins (e.g., TPM, AWS Nitro Enclaves, Kubernetes)
 - Issues X.509 or JWT SVIDs bound to hardware or environment

Cloud-native attestation protocols enable scalable, policy-driven trust in dynamic, multi-tenant environments.

Attestation failure modes:

Stale PCR Values

- PCRs may reflect a previous known-good state even if the current system is compromised
- Example:
 - Attacker boots a trusted OS, then modifies it in RAM
- **Fix:** Enforce **runtime** integrity checks, not just boot-time measurements

Attestation failure modes: Spoofed or Weak Policies

- Verifiers may accept overly broad or forged PCR policies
- Example:
 - Misconfigured verifier trusts any AK or PCR combination
- **Fix:** Define strict, signed reference values and use *policy versioning*

Attestation failure modes: Attestation Replay, Bad Bindings

- Reusing an old (valid) attestation response to gain access
 - **Fix:** Include nonces or *freshness* checks in the attestation challenge
-
- Quotes not properly bound to specific keys, workloads, or sessions
 - **Fix:** Use attestation tokens (e.g., JWT) that are cryptographically bound to the request context

Even attestation can fail if measurements are stale or policies are weak — design for freshness, tight binding, and policy control.

Section 3

Overview of Secure Key Provisioning

Key Provisioning for Cloud Computing

- Definition
 - Key provisioning is the process of securely **generating, transferring, and storing** cryptographic keys into a trusted environment (i.e., TPM, TEE).
- Cloud context:
 - Cloud services often need to protect sensitive data at rest and in use
 - Provisioned keys enable confidential workloads, secure boot, encryption services, and runtime protection

Key Provisioning in Confidential Computing

- Keys are provisioned **only if attestation succeeds**
 - Ensures keys are bound to the correct code and platform state
 - Often tied to hardware identity (e.g., SGX measurement, PCR policy, SEV-SNP report)
- Provisioning = **Building Trust Anchors**
 - Without secure key provisioning, workloads are vulnerable to:
 - Key leakage
 - Rogue administrators
 - Platform compromise

Key provisioning is a foundational step for trusted execution and encrypted computation in the cloud.

Why Hardware-Backed Key Storage Matters

- Protecting keys from software attacks:
 - Keys stored in RAM or files can be dumped, extracted, or leaked
 - Malware or compromised OS can access unprotected key material
 - Hardware-based storage provides strong isolation from host software
- What hardware provides:
 - Tamper-resistance: Keys never leave the secure hardware boundary
 - Isolation: Even privileged software (e.g., hypervisors, root) can't extract keys
 - Anti-replay: Keys are bound to platform state (e.g., PCR values, measurements)
 - Policy enforcement: Keys only usable if specific integrity conditions are met

TPM: seals keys to boot state with PCRs. SGX: bind keys to a specific enclave (MRENCLAVE).

Hardware-backed key storage turns trust assumptions into enforceable, verifiable security boundaries.

Key lifecycle: generation, wrapping, storage, usage

1. Key Generation
2. Wrapping (Sealing)
3. Secure Storage
4. Usage / Unsealing

Key lifecycle: generation, wrapping, storage, usage

1. Key Generation

- Keys are generated inside a trusted environment (TPM, TEE, HSM)
- Ensures the key material never appears in plaintext in host memory

2. Wrapping (Sealing)

3. Secure Storage

4. Usage / Unsealing

Key lifecycle: generation, wrapping, storage, usage

1. Key Generation

2. Wrapping (Sealing)

- Keys are encrypted (“wrapped”) using a device-specific root key
- Binding can include:
 - PCR values (TPM)
 - Enclave identity (SGX MRENCLAVE)
 - VM state (SEV-SNP attestation report)

3. Secure Storage

4. Usage / Unsealing

Key lifecycle: generation, wrapping, storage, usage

1. Key Generation

2. Wrapping (Sealing)

3. Secure Storage

- Wrapped keys stored on disk or in a secure key vault
- Even if exfiltrated, they are useless outside the trusted environment

4. Usage / Unsealing

Key lifecycle: generation, wrapping, storage, usage

1. Key Generation

2. Wrapping (Sealing)

3. Secure Storage

4. Usage / Unsealing

- Key can only be unwrapped and used if the system meets integrity conditions
- Typically performed in-place inside the secure hardware
- Used for:
 - Decrypting sensitive data
 - Signing, authenticating, or deriving session keys

Sealing Keys to Hardware State (1/2)

- What is *sealing*?
 - Sealing is the process of encrypting a key or data blob using a hardware-resident key.
 - For TPM, sealing is bound to specific PCR values (Platform Configuration Registers)

Sealing Keys to Hardware State (2/2)

- It works in 3 steps:

1. Measurement phase

- TPM records boot-time measurements into PCRs (e.g., BIOS, bootloader, kernel)

2. Sealing Phase

- Data is encrypted with a TPM key that's only usable if PCRs match their recorded state

3. Unsealing Phase

- On boot, if the system state matches (i.e., trusted boot chain), TPM allows decryption
- Otherwise, key remains inaccessible

What does sealing with hardware state prevents?

- Ensures secrets are tied to a specific software/hardware configuration
- Prevents use of sealed keys if:
 - Malware has altered boot process
 - Unauthorized OS or kernel is loaded
 - The device is cloned or tampered with

Secure boot + key provisioning

- A layering approach

I. Secure Boot: Rooting Trust in the Platform

- Establishes a chain of trust from firmware → bootloader → kernel → OS
- Each stage is measured and verified before the next executes
- Ensures system starts in a known-good state

Secure boot + key provisioning

- A layering approach

I. Secure Boot: Rooting Trust in the Platform

- Establishes a chain of trust from firmware → bootloader → kernel → OS
- Each stage is measured and verified before the next executes
- Ensures system starts in a known-good state

Result: trusted foundation for further security operations

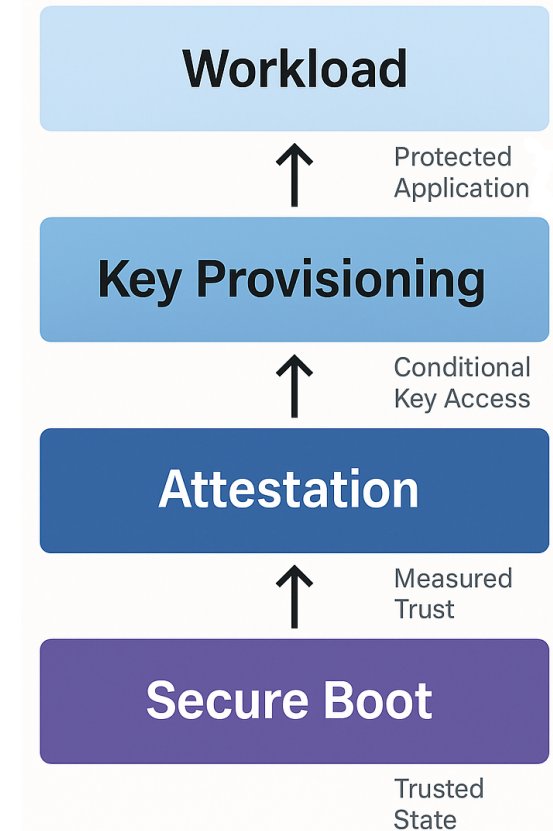
Secure boot + key provisioning

- A layering approach
2. Key Provisioning: trusting workloads only on secure platforms
- Secrets or encryption keys are released only after attestation succeeds
 - Attestation proves that system measurements match expected values
 - Keys are sealed to TPM PCRs or TEE identity (e.g., SGX MRENCLAVE)

Sensitive operations are allowed only if system integrity is verified

Secure boot + key provisioning

- Layered Trust Model
 - Provisioning → Conditional Key Access
 - Attestation → Measured Trust
 - Secure Boot → Trusted State

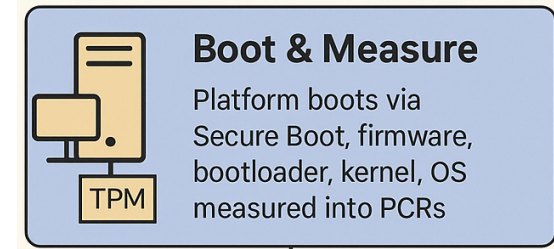


This layered approach creates a high-assurance environment: keys and sensitive operations are only permitted if the platform is provably trustworthy.

Secure provisioning flow

I. Boot & Measure

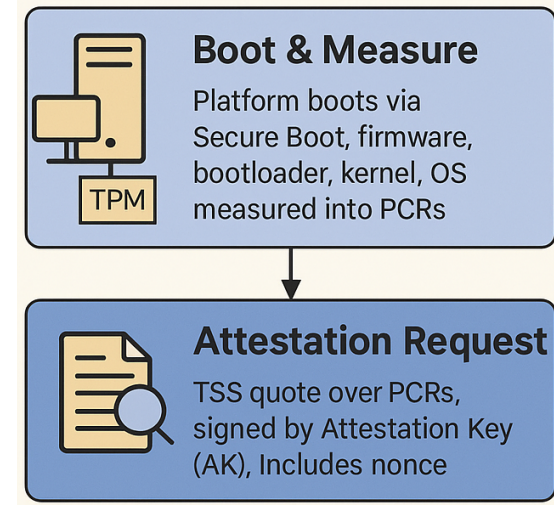
- Platform boots via Secure Boot
- Firmware, bootloader, kernel, and OS are measured into PCRs



Secure provisioning flow

2. Attestation Request

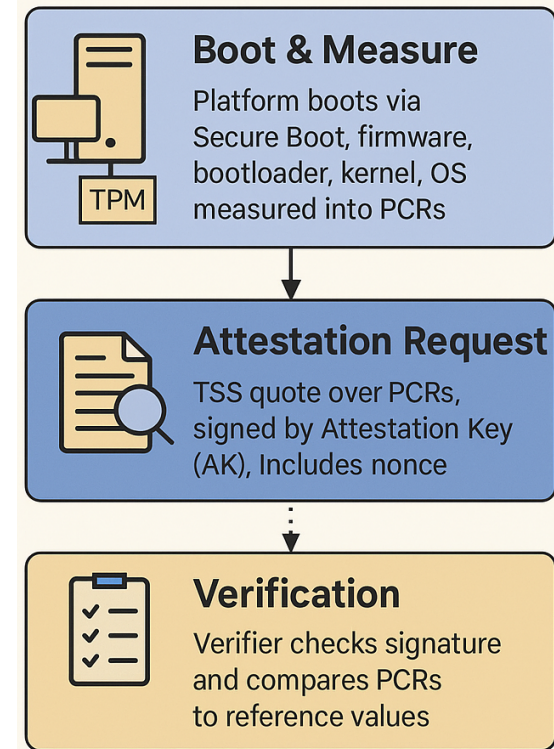
- The system generates a TPM quote over selected PCRs
- Signed using the Attestation Key (AK)
- Quote includes nonce to prevent replay



Secure provisioning flow

3. Verification

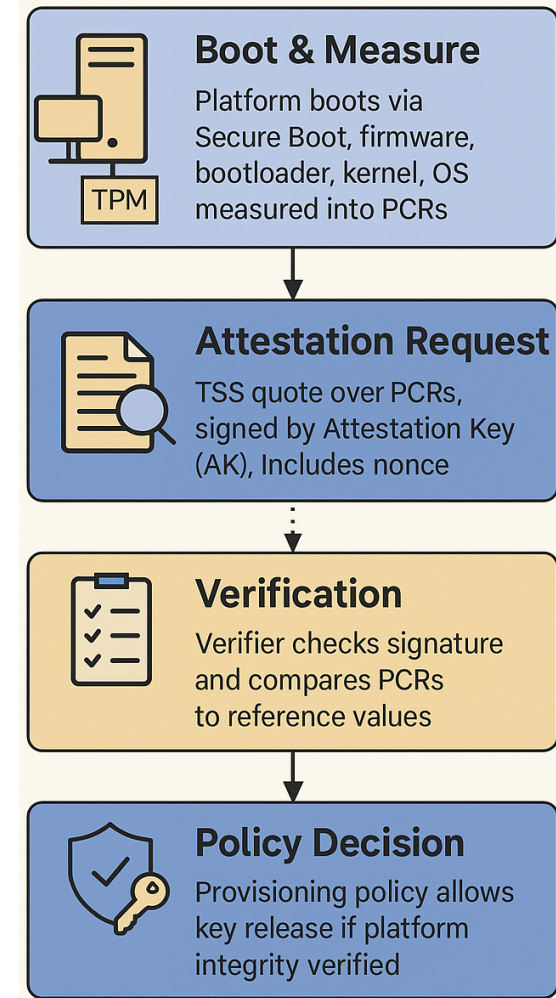
- Remote verifier checks:
- Signature (using EK → AK cert chain)
- PCR values match expected reference measurements



Secure provisioning flow

4. Policy Decision

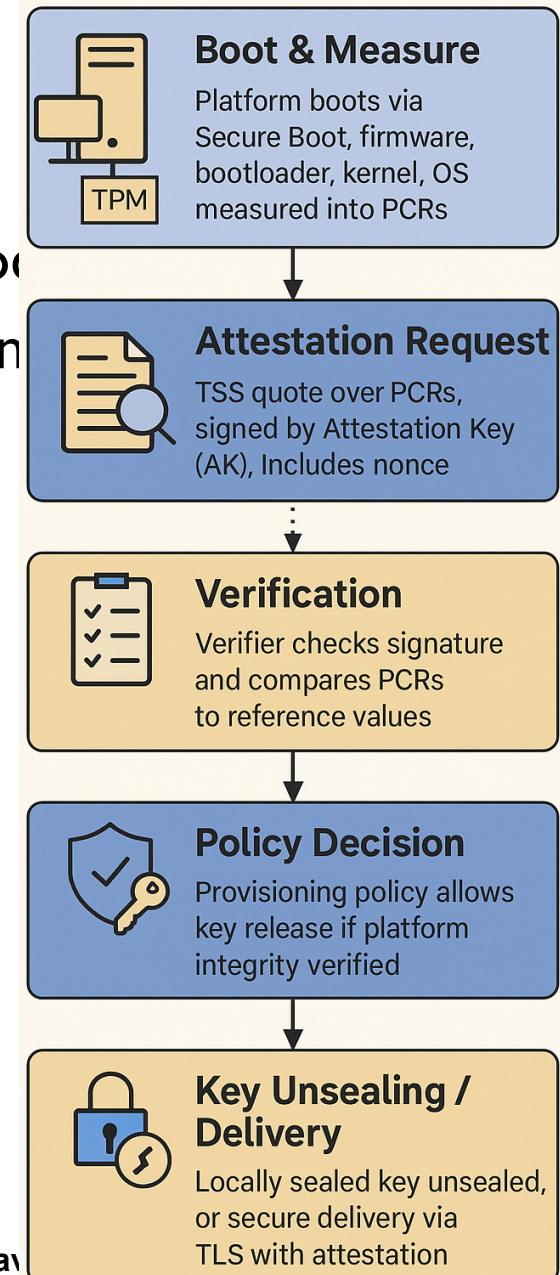
- If platform integrity is verified, provisioning policy allows key release
- Example: “Only provision keys to devices running signed OS X with secure boot enabled”



Secure provisioning flow

5. Key Unsealing / Delivery

- TPM unseals locally sealed key (e.g., BitLocker)
- Or, a secure channel (TLS with attestation) to deliver a key to the platform



Enclave key provisioning (high-level preview)

- What is an enclave? (*short recap from chap. 1*)
 - A protected region of memory created by the CPU
 - Isolated from the OS, hypervisor, and even privileged software
 - Used in TEEs like Intel SGX, AMD SEV, and Intel TDX
- Key provisioning with enclaves, in 3 simple steps:
 1. enclave creation
 2. remote attestation
 3. key delivery

Key Provisioning in Enclaves

Chapter 4

- Step 1 (creation):
 - Hardware verifies the enclave code and state
 - Generates a measurement (e.g., SGX's MRENCLAVE)
- Step 2 (attestation):
 - Enclave sends a signed report to a verifier
 - Verifier checks the identity and integrity of the enclave
- Step 3 (key delivery)
 - If attestation is successful, the verifier sends an encryption key
 - Key is delivered over a secure channel into the enclave only

Key Provisioning in Enclaves

- Ensures keys are only provisioned to trusted code in a verifiable and isolated environment
- Key remains protected from cloud admins, host OS, and hypervisors

We will later explore how modern TEEs (SGX, SEV, and TDX) each implement these mechanisms.

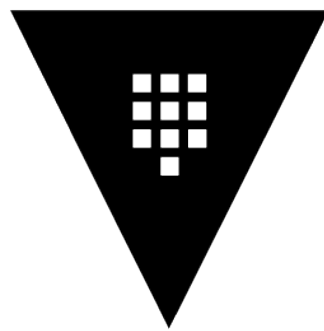
Remote provisioning & attestation

- Recap of attestation flow:
 - TEE is initialized
 - TEE generates an attestation report (quote) from:
 - platform measurements, code, nonce
 - Report is sent to a remote verifier (key owner)
 - Key owner (KMS, cloud operator) validates the quotes, and sends the key
 - Key is wrapped/encrypted for the specific TEE instance
 - Key protected in transit (TLS), public key derived from enclave identity
 - Even if intercepted, key useless outside that TEE

The key is provisioned only if attestation passes, and is cryptographically bound to the trusted environment.

Secrets management systems

- Secure services for storing, managing, and accessing secrets, such as:
 - API keys, credentials, tokens
 - Encryption keys
 - Certificates
- Let's see two real-world examples:
 - HashiCorp Vault
 - Azure Key Vault



HashiCorp

Vault

- Open-source and extensible
- Supports pluggable backends: cloud KMS, HSMs (hardware security module), hardware-backed keys
- Advanced features:
 - Dynamic secrets (e.g., DB credentials)
 - Identity-based access policies
 - Auto-unseal with TPM, HSM, or cloud KMS

HashiCorp Vault: common APIs

- For storing and retrieving **application secrets**
 - `POST /v1/secret/data/<path>`
 - Write or update secret
 - `GET /v1/secret/data/<path>`
 - Read a versioned secret
 - `DELETE /v1/secret/data/<path>`
 - Soft delete secret
- Cryptographic operations without exposing the key
 - `POST /v1/transit/encrypt/<key>` – Encrypt data
 - `POST /v1/transit/decrypt/<key>` – Decrypt data
 - `POST /v1/transit/sign/<key>` – Sign message
 - `POST /v1/transit/verify/<key>` – Verify signature
- More APIs for secret engine management, policy control, etc.



Azure Key Vault

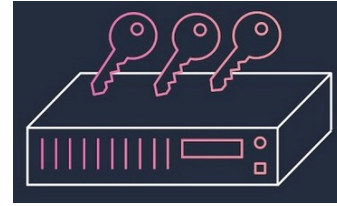
- Cloud-native secrets, key, and certificate store
 - Backed by Azure-managed HSMs or software-protected keys
 - Integrated with Azure AD and RBAC
 - Used with Azure Confidential VMs/Containers and attestation policies

Are these required ?

- Provide controlled provisioning of secrets to verified workloads
- Integrate with attestation-based policies
- Enforce fine-grained access control and auditing

Vault and Key Vault are essential for securely handling secrets in cloud and confidential computing environments.

HSM-Backed Cloud APIs



- Cloud providers increasingly expose APIs backed by Hardware Security Modules (HSMs)
 - Examples: **AWS CloudHSM**, Azure Managed HSM, Google Cloud EKM
- Features:
 - **Key material never leaves HSM boundary**
 - Integrated with cloud-native KMS and identity policies
 - Can be combined with attestation for conditional key access

Summary: HRoT in practice

- Why Roots of Trust Matter
 - They anchor the entire system's security
 - Without them, all higher assurances are fragile
- Implemented via:
 - Secure Boot – Ensures trusted startup
 - TPM/fTPM – Measures and seals state
 - Attestation – Proves trustworthiness to remote parties
 - Provisioning – Releases keys only to trusted workloads
- In Practice:
 - BitLocker, Linux Measured Boot, Cloud VMs, ...
 - Integrated with HashiCorp Vault, Azure Key Vault, HSM-backed KMS APIs
 - Critical for confidential containers, remote key provisioning, and secure automation

Key Takeaways

- Hardware-backed roots of trust provide **verifiability**, **sealing**, and **measurable integrity**
- They form the **foundation for confidential computing**

Questions & Discussion

- What new risks emerge if the root of trust is compromised?
- What's the difference between “trust” and “trustworthiness”?
- Exercise: How would you design a system that needs to revoke trust?

Revoking Trust !

- Keys or code no longer meet policy
- **Vulnerabilities** are discovered post-deployment (e.g., BIOS flaw, Spectre)
- Platform or TEE is compromised
- Lifecycle events: decommissioning, credential rotation

Revoking Trust ! (2/2)

- Key mechanisms in system design to support revoking trust:

1. Versioned Reference Measurements

- Attestation checks specific code versions (hashes)
- Revoke older/unsafe MRENCLAVE (SGX) or PCR (TPM) values

2. Dynamic Policy Enforcement

- Maintain allow-lists / deny-lists of acceptable measurements
- Use a centralized verifier or policy engine (e.g., SPIRE, Key Broker)

3. Short-Lived Credentials

- Provision ephemeral secrets or session keys
- Require frequent **re-attestation** to renew access
 - “continuous attestation” is a hot research topic

4. Remote Attestation Fencing

- Disallow provisioning if the attestation reflects:
- Revoked firmware or bootloader
- Known-vulnerable enclave version